Sensor-Based IoT Simulation: Temperature-Controlled LED & Fan System

CodeAlpha IoT Internship — Task 2: Sensor-Based Simulation (Tinkercad)

1. Project Overview

This project simulates a simple IoT temperature-monitoring node using a DHT11 temperature sensor and an Arduino Uno on Tinkercad Circuits. When the measured temperature rises above a set threshold, the Arduino automatically turns on a red LED (warning indicator) and a small DC motor representing a cooling fan. When the temperature drops back below the threshold, both are switched off. This mirrors a real IoT use case: automated climate control in a server room, greenhouse, or smart home, where a microcontroller reacts to sensor data without needing manual input.

2. Components Used (Tinkercad)

• Arduino Uno R3
• DHT11 Temperature & Humidity Sensor
• Red LED (warning indicator) + 220Ω resistor
• Green LED (normal-status indicator) + 220Ω resistor
• Small DC Motor (fan) with NPN transistor (2N2222) + 1N4007 flyback diode + 1kΩ base resistor
• Breadboard and jumper wires

3. Circuit Description

The DHT11's data pin connects to Arduino digital pin 2. The red LED's anode connects through a 220Ω resistor to digital pin 8, and the green LED's anode connects through a 220Ω resistor to digital pin 7; both cathodes go to ground. The DC motor is driven indirectly: its positive lead connects to the 5V rail, its negative lead connects to the transistor's collector, the transistor's emitter goes to ground, and its base connects through a 1kΩ resistor to Arduino digital pin 9 (the Arduino cannot drive a motor directly, so the transistor acts as a switch). A flyback diode is placed across the motor terminals to protect the transistor from voltage spikes when the motor switches off. In Tinkercad, the DHT11's temperature can be adjusted manually on the component to simulate rising and falling temperature.

4. Arduino Code

```
#include <DHT.h>

#define DHTPIN 2
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
```

```
const int redLED   = 8;   // warning indicator
const int greenLED = 7;   // normal status indicator
const int fanPin    = 9;  // transistor base -> fan
const float TEMP_THRESHOLD = 30.0; // degrees Celsius

void setup() {
  pinMode(redLED, OUTPUT);
  pinMode(greenLED, OUTPUT);
  pinMode(fanPin, OUTPUT);
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  float temperature = dht.readTemperature();

  if (isnan(temperature)) {
    Serial.println("Sensor read failed");
    delay(2000);
    return;
  }

  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println(" C");

  if (temperature > TEMP_THRESHOLD) {
    digitalWrite(redLED, HIGH);
    digitalWrite(greenLED, LOW);
    digitalWrite(fanPin, HIGH);   // turn fan ON
    Serial.println("Status: HOT -> Fan ON, Warning LED ON");
  } else {
    digitalWrite(redLED, LOW);
    digitalWrite(greenLED, HIGH);
    digitalWrite(fanPin, LOW);    // turn fan OFF
    Serial.println("Status: NORMAL -> Fan OFF");
  }

  delay(2000); // DHT11 needs ~2s between readings
}
```

5. Code Explanation

The sketch uses the standard DHT sensor library to read temperature from the DHT11 every two seconds, which is the minimum safe polling interval for this sensor. Three digital outputs are configured: two LEDs for visual status and one pin driving the transistor that switches the fan motor. On each loop iteration, the code checks whether the temperature exceeds a fixed threshold (30°C in this simulation). If it does, the red warning LED and the fan are switched on and the green LED turns off; otherwise the system reverts to its normal (green LED only) state. A basic error check using isnan() handles the case where the sensor briefly fails to return a valid reading, which can happen with the DHT11 even in simulation. All state changes are also printed to the Serial Monitor, so the behavior can be verified in real time without needing to watch the components directly.

6. How This Was Simulated in Tinkercad

1. Create a new Tinkercad Circuits project and add an Arduino Uno, a DHT11 sensor, two LEDs with 220Ω resistors, a small DC motor, an NPN transistor, a 1N4007 diode, and a breadboard.
2. Wire the components according to Section 3 above.
3. Open the code editor in Text mode and paste in the sketch from Section 4.
4. Start the simulation. While it runs, click the DHT11 component and drag its temperature slider above and below 30°C to observe the red LED and fan turning on and off, and the green LED taking over when the temperature is normal.
5. Take screenshots of (a) the completed circuit, (b) the code editor, and (c) the Serial Monitor output showing both the "HOT" and "NORMAL" states, and include them alongside this document in your submission folder as required by the task instructions.

7. Real-World Application

This exact pattern — a sensor triggering an actuator through a microcontroller once a threshold is crossed — is the foundation of many real IoT deployments: server-room cooling systems, greenhouse climate control, industrial equipment overheat protection, and smart-home HVAC automation. In a production deployment, the Arduino would typically also publish sensor readings to a cloud dashboard over Wi-Fi (e.g., using an ESP32 and the MQTT protocol) so the status could be monitored remotely, in addition to the local LED/fan response demonstrated in this simulation.

Note

Screenshots of the running Tinkercad simulation and Serial Monitor output should be added by you before final submission, since they must be captured directly from your own Tinkercad session.